

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы.....	4
1.2. Документы, на основании которых ведется разработка.....	4
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение.....	Error! Bookmark not defined.
2.3. Краткая характеристика области применения программы	5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	6
3.1. Постановка задачи на разработку программы	6
3.2. Описание функционирования программы.....	7
3.2.1. Описание архитектуры приложения	7
3.2.2. Описание реализации авторизации	8
3.2.3. Описание реализации взаимодействия с базой данных	9
3.2.4. Описание маппинга моделей	10
3.2.5. Описание реализации бизнес-сервисов	10
3.2.6. Описание реализации валидации данных	11
3.2.7. Описание реализации инфраструктуры	12
3.2.8. Описание реализации конфигурации.....	12
3.2.9. Описание реализации API.....	13
3.2.10. Описание реализации интеграции LLM	Error! Bookmark not defined.
3.2.11. Описание работы с Docker	14
3.3. Описание и обоснование выбора состава технических и программных средств.....	14
3.3.1. Состав технических и программных средств	14
3.3.2. Обоснование выбора состава технических и программных средств	15
3.4. Описание метода организации входных и выходных данных.....	15
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	16
4.1. Ориентировочная экономическая эффективность	16
4.2. Предполагаемая потребность	16
4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами.....	16

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ИСПОЛЬЗОВАННЫЕ ИСТОЧНИКИ.....	17
ПРИЛОЖЕНИЕ 1	19
ПРИЛОЖЕНИЕ 2	20
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	22

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «iOS-приложение для организации встреч с друзьями “Совпало”».

Условное обозначение – «Sovpalo».

1.2. Документы, на основании которых ведется разработка

Основанием для разработки являются учебный план подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и тема курсового проекта, утвержденная академическим руководителем.

Документ подготовлен в рамках образовательной программы «Программная инженерия» факультета компьютерных наук НИУ «Высшая школа экономики».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Функциональное назначение

Программа представляет собой серверную часть приложения «Sovpalo», предназначенного для общения и взаимодействия пользователей внутри компаний и сообществ. Сервер обеспечивает работу профилей, компаний, встреч, посещаемости, идей, встроенного чата и генерации вариантов идей с использованием LLM, а также отвечает за хранение и обработку данных клиентского приложения.

2.2. Эксплуатационное назначение

Приложение «Sovpalo» используется для общения пользователей, объединения в компании, приглашения новых участников, планирования встреч, обмена сообщениями во встроенном чате, публикации идей и согласования удобного времени для совместной активности. Серверная часть обеспечивает выполнение этих сценариев и предоставляет клиентскому приложению необходимые данные и интерфейсы.

2.3. Краткая характеристика области применения программы

Программа применяется в составе цифрового сервиса для дружеского и неформального взаимодействия между людьми в рамках небольших сообществ, компаний по интересам и команд. Она используется в ситуациях, когда необходимо поддерживать общение, совместно планировать встречи, обсуждать идеи, согласовывать время участников и вести переписку в одном пространстве.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Целью разработки является создание серверной части приложения «Sovpalo», предназначенной для организации общения, координации совместной активности и взаимодействия пользователей в рамках компаний и сообществ.

- 1) Аутентификация и авторизация пользователей, включая регистрацию и вход по электронной почте, восстановление пароля, вход через аккаунт Apple и аутентификацию через Telegram OAuth.
- 2) Управление пользовательским профилем, включая получение данных текущего пользователя, загрузку и удаление аватара, а также удаление учетной записи.
- 3) Управление компаниями, включая создание, редактирование, удаление и просмотр информации о компаниях.
- 4) Управление составом участников компаний, включая приглашение пользователей, принятие и отклонение приглашений, просмотр списка участников, удаление участников, выход из компании и передачу прав владельца.
- 5) Организация встреч и совместных мероприятий, включая создание, редактирование, удаление и просмотр встреч личного и внутрикорпоративного формата.
- 6) Учет посещаемости мероприятий с возможностью фиксировать статус участия пользователей и получать сводную информацию по посещаемости.
- 7) Управление идеями внутри компании, включая создание, редактирование, просмотр и оценку идей с помощью механизма лайков.
- 8) Координация времени участников через добавление, изменение и удаление интервалов доступности, просмотр общей доступности и поиск пересечений по времени.
- 9) Обеспечение обмена сообщениями между участниками компаний через встроенный чат, включая обновление данных в реальном времени.
- 10) Предоставление REST API для взаимодействия серверной части с клиентским приложением, а также WebSocket-интерфейса для работы чата.
- 11) Организация хранения и обработки данных с использованием PostgreSQL, а также применение Redis для вспомогательных серверных сценариев.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- 12) Обеспечение развертывания и сопровождения приложения с использованием Docker, миграций базы данных и интеграции с LLM-провайдером для генерации идей.

3.2 Описание функционирования программы

Серверная часть приложения «Sovpalo» разработана на языке Go с использованием веб-фреймворка Gin. Она принимает и обрабатывает запросы клиентского приложения, выполняет бизнес-логику и обеспечивает основные сценарии системы: работу с компаниями, встречами, посещаемостью, идеями, чатом и генерацией вариантов идей с помощью LLM.

3.2.1 Описание архитектуры приложения

Одной из первых задач при разработке проекта был выбор архитектуры серверной части. Требовалось организовать код так, чтобы в дальнейшем можно было добавлять новый функционал без роста связности между модулями. Поскольку в приложении используются компании, приглашения, встречи, лайки, интервалы доступности, чат и генерация идей, отсутствие четкой структуры быстро усложнило бы сопровождение.

- 1) На этапе проектирования рассматривались несколько вариантов. Первый вариант предполагал размещение большей части логики прямо в HTTP-обработчиках с прямым обращением к базе данных. Такой подход допустим для небольших прототипов, но по мере роста числа сценариев делает обработчики перегруженными. Второй вариант предполагал более глубокую декомпозицию системы на отдельные сервисы и дополнительные уровни, однако для курсового проекта такой подход оказался избыточным.
- 2) В результате был выбран многослойный backend с разделением на handler, service, repository и model. Такое решение позволяет четко разделить ответственность между частями системы. HTTP-слой принимает запрос и формирует ответ, service-слой содержит прикладную логику, repository-слой работает с хранилищами данных, а модели задают структуру сущностей и контрактов обмена данными.
- 3) **Handler-слой** выполняет разбор JSON, multipart/form-data и параметров маршрутов. **Service-слой** проверяет права, собирает данные из нескольких источников, определяет допустимость действия и вызывает нужные репозитории.

Repository-слой отвечает только за чтение и запись данных и не содержит логики

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

пользовательских сценариев. За счет этого один и тот же репозиторий может использоваться в нескольких сервисах без дублирования прикладного кода.

- 4) Преимуществом выбранного решения стало то, что после добавления новых сценариев общая структура проекта не потребовала пересмотра. Чат и генерация идей через LLM были встроены в существующую архитектуру без нарушения ранее принятого разделения ответственности. Это подтверждает, что выбранная схема соответствует масштабу и задачам проекта.
- 5) При необходимости архитектура может быть дополнительно формализована, например за счет более строгого выделения уровня use case и контрактов между слоями. Однако для текущего объема проекта выбранная структура является достаточной и обеспечивает разумный баланс между архитектурной строгостью и сложностью реализации.

3.2.2 Описание реализации авторизации

- 1) Регистрация и вход по электронной почте.

При регистрации пользователь передает адрес электронной почты, имя пользователя и пароль. После первичной проверки формируется временный код подтверждения, который отправляется по электронной почте через SMTP. Данные незавершенной регистрации сохраняются в Redis с ограниченным временем жизни. После подтверждения кода создается учетная запись пользователя и выдается JWT-токен для дальнейшей работы с API.

- 2) Авторизация через внешние провайдеры.

Дополнительно в приложении поддерживаются альтернативные способы входа: через аккаунт Apple и через Telegram OAuth. После проверки данных, полученных от внешнего провайдера, сервер либо связывает вход с существующей учетной записью, либо создает новую запись пользователя, после чего клиентскому приложению также выдается JWT-токен.

- 3) Авторизация по JWT.

Для доступа к защищенным маршрутам используется схема Bearer Token. Middleware извлекает токен из заголовка Authorization, проверяет его и определяет идентификатор текущего пользователя, который затем используется в дальнейшей обработке запроса.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4) Восстановление пароля.

При восстановлении пароля пользователь получает код подтверждения на электронную почту. После успешной проверки кода сервер позволяет задать новый пароль, который проходит дополнительную проверку и сохраняется в базе данных в виде хэшированного значения с солью.

Таким образом, подсистема авторизации объединяет несколько способов входа, проверку одноразовых кодов, безопасную работу с учетными данными и выдачу JWT-токенов. Это позволяет обеспечить доступ к приложению при сохранении единого механизма защиты API.

3.2.3 Описание реализации взаимодействия с базой данных

В приложении используются взаимосвязанные сущности: пользователь может состоять в нескольких компаниях, компании содержат участников, встречи, идеи и сообщения чата, а также связаны с приглашениями, лайками, интервалами доступности и отметками прочтения. Для такой предметной области требовалось хранилище, которое надежно поддерживает связи между сущностями и транзакционную целостность данных.

На этапе выбора подхода рассматривались разные варианты работы с данными. Можно было использовать ORM и переложить значительную часть работы на библиотеку высокого уровня либо, наоборот, писать все запросы вручную без дополнительной структуры. Первый вариант упрощает начальную разработку, но затрудняет явный контроль над запросами. Второй вариант без отдельного слоя доступа к данным приводит к дублированию SQL-кода и смешению прикладной логики с операциями хранения.

В результате был выбран следующий подход: PostgreSQL используется как основное хранилище, подключение выполняется через pgx и pgxpool, а поверх этого выделен слой репозитория, который отделяет прикладную логику от конкретных SQL-запросов. Это решение позволяет контролировать запросы и одновременно не допускать их появления в обработчиках или сервисах. Отдельные репозитории обслуживают авторизацию, компании, встречи, доступность, идеи и чат, что упрощает сопровождение и локальные изменения.

Дополнительно в проекте используется Redis, но не как универсальное вторичное хранилище, а как средство для работы с временными данными авторизации и кодами подтверждения. Изменение схемы базы данных выполняется через миграции goose. Подключение к PostgreSQL и Redis вынесено в internal/db, после чего готовые соединения передаются в репозитории. В результате сценарии приложения не зависят от деталей инициализации соединений, а работа с данными остается сосредоточенной в одном уровне архитектуры.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.4 Описание маппинга моделей

Модели предметной области представлены структурами Go с тегами db и json. Это позволяет разделить форму хранения данных и форму их передачи через API. В проекте отдельно описаны сущности User, Company, Event, Idea, ChatMessage и другие структуры предметной области, а также типы для входных и выходных данных. Благодаря этому по используемому типу обычно сразу понятно, относится ли он к записи в базе данных, телу запроса или структуре ответа.

3.2.5 Описание реализации бизнес-сервисов

Сервисный слой был выделен в проекте потому, что многие пользовательские сценарии не сводятся к одному обращению к базе данных. При работе с компаниями необходимо учитывать состав участников, права владельца и допустимость управленческих действий. При обработке встреч требуется не только сохранять поля сущности, но и определять права доступа, учитывать посещаемость и формировать вычисляемые характеристики встречи. Аналогичная ситуация возникает при работе с идеями, которые могут создаваться вручную, оцениваться пользователями и генерироваться с использованием LLM. Если бы эта логика оставалась в обработчиках, HTTP-слой стал бы слишком сложным, а перенос ее в репозитории смешал бы прикладные правила с операциями хранения данных.

На этапе проектирования рассматривались два основных подхода. Первый подход заключался в минимизации сервисного слоя, когда большая часть прикладных решений принимается либо в HTTP-обработчиках, либо в SQL-запросах и репозиториях. Такой вариант сокращает количество промежуточных компонентов, но при росте числа сценариев затрудняет повторное использование логики и усложняет тестирование. Второй подход предполагал выделение самостоятельных сервисов, каждый из которых отвечает за определенную подсистему и координирует обращения к репозиториям, проверкам прав доступа, генерации файловых данных и внешним интеграциям. Для данного проекта был выбран именно второй вариант, так как он лучше подходит для расширяемой серверной части с несколькими взаимосвязанными доменными сценариями.

В текущей реализации это выражается в наличии сервисов Authorization, Company, Event, Availability, Idea и Chat. Каждый из них получает только необходимые зависимости и выполняет полный прикладной сценарий: от предварительных проверок и подтверждения членства пользователя в компании до записи данных и дополнительных операций, связанных с файлами, генерацией идей или обработкой сообщений. Например, сервис встреч отвечает не только за

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

создание и изменение событий, но и за расчет признаков встречи, используемых в отдельном маршруте features. Сервис идей объединяет ручную работу с идеями, лайки и обращение к генератору черновиков, а сервис чата включает проверку прав, обработку вложений, удаление сообщений и фиксацию факта прочтения.

Выбранное решение подтвердило свою практическую пригодность в ходе развития проекта. После добавления встроенного чата и интеграции с LLM не потребовалось перерабатывать общий каркас приложения: новые сценарии были встроены в существующую сервисную модель и использовали те же принципы декомпозиции. Это позволило сохранить обработчики сравнительно компактными, а репозитории ограничить задачами доступа к данным. В дальнейшем сервисный слой может быть развит за счет более формального выделения use-case объектов и расширения тестового покрытия, однако уже в текущем виде он обеспечивает достаточную читаемость, изоляцию ответственности и удобство сопровождения.

3.2.6 Описание реализации валидации данных

Валидация входных данных организована в несколько уровней, поскольку одной проверки формата запроса недостаточно. Корректный JSON сам по себе не гарантирует допустимость операции и корректность значений в контексте компании, встречи или чата.

- 1) На первом уровне используется Gin, который разбирает тело запроса, path-параметры и базовые ограничения, заданные через binding. Здесь отсеиваются пустые обязательные поля, некорректные email-адреса и другие случаи, при которых запрос не может быть корректно обработан.
- 2) Следующий уровень расположен в сервисном слое. Здесь выполняются прикладные проверки: допустимость смены владельца компании, корректность статуса посещаемости, наличие текста в сообщении, соблюдение лимита файлов в чате и допустимый диапазон количества идей для генерации.
- 3) Последний уровень связан с данными. На нем проверяются существование необходимых записей, факт участия пользователя в компании и соблюдение ограничений целостности. Такое разделение позволяет не смешивать ошибки формата запроса, прикладных правил и состояния данных.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.7. Описание реализации инфраструктуры

Инфраструктурная часть проекта не является большой по объему, однако без ее формализации приложение представляло бы собой набор файлов без понятной схемы запуска. На этом этапе требовалось определить структуру директорий, порядок инициализации зависимостей, использование health-check, миграций и воспроизводимого окружения.

- 1) `cmd` — точки входа для запуска API-сервера и модуля миграций.
- 2) `internal` — конфигурация приложения и инициализация подключений к PostgreSQL и Redis.
- 3) `pkg` — обработчики HTTP-запросов, сервисы, репозитории и модели предметной области.

Маршруты приложения объединяют открытые и защищенные сценарии, а загруженные файлы публикуются через отдельный HTTP-маршрут. Здесь же находится логика health-check, позволяющая отдельно проверить сам сервис, SMTP и доступность хранилищ. Хотя этот блок менее заметен, чем чат или генерация идей, именно он обеспечивает удобство запуска, проверки и сопровождения системы.

Точка входа собирает инфраструктуру последовательно: создаются подключения к PostgreSQL и Redis, затем инициализируются health-репозитории, после чего формируются основные репозитории, сервисы и обработчики. Такой порядок упрощает чтение кода и делает процесс сборки сервера понятным.

Развертывание через Docker Compose решает задачу воспроизводимости окружения. В compose-файле описаны PostgreSQL, Redis, миграции и API, поэтому проект может быть развернут без ручной настройки каждого компонента по отдельности.

3.2.8. Описание реализации конфигурации

Конфигурация вынесена в отдельный модуль и считывается из `.env` и переменных окружения. Такой подход позволяет не распределять параметры по разным частям кода, что особенно важно при наличии настроек для базы данных, Redis, SMTP, JWT и LLM-провайдера.

Конфигурация включает порт приложения, параметры PostgreSQL и Redis, SMTP-настройки, секрет JWT и соль пароля, а также переменные для генерации идей через YandexGPT. Если часть

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

параметров отсутствует, используются значения по умолчанию. Это упрощает локальный запуск и уменьшает количество ручных действий при развертывании.

3.2.9. Описание реализации API

В проекте используется REST API, поскольку для данного набора сценариев это наиболее естественный способ взаимодействия клиента и сервера. Клиентскому приложению требуются маршруты для работы с пользователями, компаниями, участниками, встречами, идеями, доступностью и сообщениями. Для чата дополнительно используется WebSocket-канал, который позволяет доставлять новые сообщения и события чтения без постоянного опроса сервера.

3.2.10. Описание реализации LLM и чата

Интеграция LLM и реализация чата были важными задачами на позднем этапе разработки. Их требовалось встроить в уже существующую систему так, чтобы новые возможности не нарушили принятое разделение ответственности между слоями и не потребовали переработки базовых сценариев приложения.

При добавлении генерации идей рассматривался вопрос о том, насколько глубоко модель должна быть включена в предметную логику. Один вариант предполагал генерацию текста прямо в обработчике с немедленным сохранением результата как обычной идеи. Второй вариант заключался в выносе генерации в отдельный сервис и сохранении результата в виде черновиков, которые пользователь может просмотреть и при необходимости сохранить. Был выбран второй подход, поскольку в таком случае языковая модель не принимает решение за пользователя, а используется как инструмент подготовки вариантов. В проекте это реализовано через IdeaService, который собирает контекст, формирует prompt, вызывает YandexGPT и возвращает список подготовленных вариантов.

Отдельное внимание было уделено формату ответа. Результат генерации приводится к структуре GeneratedIdeaDraft, содержащей заголовок, описание, источник и prompt. Такое представление упрощает работу как на серверной, так и на клиентской стороне, поскольку позволяет обрабатывать результат модели как обычный набор данных, а не как произвольный текст.

Реализация чата требовала быстрого обмена сообщениями между участниками компании и безопасной работы с вложениями. Поэтому был выбран подход, при котором история сообщений и сведения о прочтении хранятся в PostgreSQL, а актуализация клиентских данных выполняется через WebSocket. В результате чат встроен в общую модель компаний, использует существующую

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

систему прав доступа и поддерживает отправку текста, вложений, удаление собственных сообщений, отметку о прочтении и расчет числа непрочитанных сообщений.

3.2.11. Описание работы с Docker

Docker и Docker Compose используются в проекте как средство обеспечения воспроизводимого окружения. Поскольку система включает базу данных, Redis, миграции и API, их ручной запуск по отдельности быстро становится неудобным и повышает вероятность ошибок окружения.

В compose-файле описаны отдельные сервисы для PostgreSQL, Redis, миграций и API. Для контейнера API также проброшены переменные LLM-провайдера и YandexGPT, поэтому сценарий генерации идей не требует дополнительной ручной настройки на конкретной машине.

Такой способ запуска позволяет сразу увидеть состав проекта в рабочем виде, порядок запуска зависимостей и требования к готовности отдельных компонентов до старта приложения.

3.3. Описание и обоснование выбора состава технических и программных средств

3.3.1. Состав технических и программных средств

Программные средства:

Поддерживаемые платформы:

- Linux (рекомендуется для серверного развертывания и контейнерного запуска).
- macOS 12 и выше.
- Windows 10/11 при использовании Docker Desktop или WSL2.

Технические средства:

- Процессор: не менее 2 вычислительных ядер.
- Оперативная память: не менее 4 ГБ.
- Свободное дисковое пространство: не менее 2 ГБ для приложения, зависимостей и контейнеров.
- Сетевое подключение: требуется для загрузки контейнеров, работы SMTP и внешних провайдеров аутентификации.
- Рекомендуемая конфигурация: 4 ядра и более при одновременной работе нескольких сервисов и параллельной обработке запросов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.3.2. Обоснование выбора состава технических и программных средств

Выбор состава технических и программных средств в рамках данного проекта не требовал специальной инфраструктуры, однако было необходимо соотнести используемый стек с реальными условиями эксплуатации и разработки. Серверное приложение должно стабильно работать с HTTP-запросами, подключением к PostgreSQL и Redis, выполнением миграций, обработкой файловых вложений и обращением к внешнему LLM-провайдеру.

По этой причине в документе зафиксированы умеренные, но реалистичные требования к памяти, диску и сети. Это особенно важно для локальной разработки, когда на одной машине одновременно работают контейнеры базы данных, Redis, модуль миграций и API-сервер, а также могут выполняться загрузка файлов и обращения к внешним сервисам.

В результате был определен набор требований, который соответствует фактическому стеку проекта и подходит для его стабильной эксплуатации.

3.4. Описание метода организации входных и выходных данных

Входные данные поступают в систему в нескольких формах. К ним относятся JSON, path- и query-параметры, HTTP-заголовки, а в сценариях загрузки файлов — multipart/form-data. После получения запрос преобразуется в структуру входных данных, проходит синтаксическую и прикладную валидацию, а затем передается в сервисный слой. Выходные данные в основном формируются в формате JSON. Для отдельных сценариев используются специальные структуры ответа, например объект с полем items при генерации вариантов идей и отдельный маршрут вычисляемых характеристик встречи.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Курсовой проект не включает расчет экономической эффективности.

4.2. Предполагаемая потребность

Приложение «Sovpalo» ориентировано на небольшие сообщества, студенческие объединения, компании по интересам и другие группы пользователей, которым требуется единая цифровая среда для общения, планирования встреч, обсуждения идей и координации совместной активности.

4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами

В качестве аналогов были выбраны сервисы, используемые для общения, координации и организации совместной активности. Они не являются полными функциональными копиями разрабатываемого приложения, однако позволяют сопоставить отдельные сценарии использования и оценить преимущества объединения нескольких функций в одной системе.

В качестве приложений-аналогов рассмотрены Telegram (1), Discord (2), Max (3), Meetup (4), TimeTree (5), Partiful (6), VK Мессенджер/чаты (7). Сравнительная таблица приведена в приложении 1.

По сравнению с указанными аналогами приложение «Sovpalo» имеет ряд особенностей, значимых для выбранной области применения.

- общение, встречи, посещаемость и идеи объединены в одном приложении;
- механизм компаний и приглашений позволяет поддерживать закрытые сообщества с контролируемым составом участников;
- генерация идей с использованием LLM ускоряет подготовку вариантов совместной активности;
- интервалы доступности и встроенный чат обеспечивают не только обсуждение, но и координацию времени участников.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.105-78 Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.101-77 Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77 Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.106-78 Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78 Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.201-78 Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.603-78 Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.102-77 Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.604-78 Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79 Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. Gin Web Framework [Электронный ресурс]. URL: <https://gin-gonic.com/en/docs/>.
14. Docker [Электронный ресурс]. URL: <https://docs.docker.com/>.
15. SMTP [Электронный ресурс]. URL: <https://pkg.go.dev/net/smtp>.
16. HTTP [Электронный ресурс]. URL: <https://pkg.go.dev/net/http>.
17. PostgreSQL [Электронный ресурс]. URL: <https://www.postgresql.org/docs/>.
18. Redis [Электронный ресурс]. URL: <https://redis.io/docs/latest/>.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

19. Go [Электронный ресурс]. URL: <https://go.dev/doc/>.
20. pgx [Электронный ресурс]. URL: <https://pkg.go.dev/github.com/jackc/pgx/v5>.
21. JSON [Электронный ресурс]. URL: <https://pkg.go.dev/encoding/json>.
22. go-redis [Электронный ресурс]. URL: <https://pkg.go.dev/github.com/redis/go-redis/v9>.
23. JWT [Электронный ресурс]. URL: <https://pkg.go.dev/github.com/dgrijalva/jwt-go>.

ПРИЛОЖЕНИЕ 1

СРАВНИТЕЛЬНАЯ ТАБЛИЦА АНАЛОГОВ

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Характеристика	1)	2)	3)	4)	5)	6)	7)	Sovpalo
Регистрация	+	+	+	+	+	+	+	+
Создание компаний/сообществ	+	+	+	+	-	-	+	+
Внутренний чат	+	+	+	-	+	+	+	+
Планирование встреч	-	+	-	+	+	+	-	+
Отметка посещаемости	-	+	-	+	-	+	-	+
Обмен идеями	-	-	-	-	-	-	-	+
Координация доступности	-	-	-	-	+	-	-	+
Несколько способов входа	-	-	-	-	-	-	-	+
Приглашения участников	+	+	+	+	+	+	+	+
Аватары и изображения	+	+	+	+	-	+	+	+

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.06-01 ТЗ 02-1				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

ТЕРМИНОЛОГИЯ

Термин	Определение
Backend	Серверная часть приложения, обрабатывающая логику, данные и взаимодействие с базой данных.
REST (Representational State Transfer)	Архитектурный стиль для веб-сервисов, основанный на стандартных HTTP-методах.
API (Application Programming Interface)	Интерфейс для взаимодействия между программными компонентами.
REST API	API, соответствующий принципам REST, использующий HTTP-запросы для обмена данными.
HTTP (HyperText Transfer Protocol)	Протокол передачи данных в сети, основа для взаимодействия клиента и сервера.
Архитектурный слой	Логически выделенный компонент системы (например, контроллеры, сервисы, репозитории).
Контейнеризация	Упаковка приложения и его зависимостей в изолированную среду (контейнер).
SQL (Structured Query Language)	Язык для управления реляционными базами данных (создание, изменение, выборка).
Миграция	Процесс изменения структуры БД с сохранением целостности данных.
Маппинг	Преобразование данных между форматами (например, из SQL-таблицы в объект приложения).

Модель	Структура данных, отражающая сущности предметной области, например таблицы базы данных.
Контроллер	Компонент, обрабатывающий HTTP-запросы и возвращающий ответы.
Сервис	Слой бизнес-логики, реализующий ключевые функции приложения.
Репозиторий	Компонент для взаимодействия с БД (выполнение запросов, маппинг результатов).
Docker контейнер	Изолированная среда для запуска приложения с его зависимостями.
Dockerfile	Текстовый файл с инструкциями для сборки Docker-образа.

Лист регистрации изменений

[illegible]